Webcam Photo Filter: Mosaic / Pixelation

Daniel Oni | CS 5382 Computer Graphics

What I did

Starting from the webcam starter code, I left the vertex shader alone and changed only the fragment shader to apply a mosaic photo filter to the live webcam feed. The filter makes the video look like it is built out of large square tiles, and a slider on the page lets me adjust how chunky or fine those tiles are while the program is running.

How the filter works

The technique is texture coordinate quantization. In the starter, every fragment samples the webcam texture at its own interpolated vTexCoord, so the output looks like a normal video. For the mosaic effect, I take that incoming vTexCoord and snap it to the center of a grid cell before the texture lookup. All fragments that land in the same cell end up reading the exact same texel from the webcam frame, which is what produces the big-pixel look.

The key line inside the fragment shader is:

```
vec2 snappedCoord = (floor(vTexCoord * uBlocks) + 0.5) / uBlocks;
fColor = vec4( texture( texturevid, snappedCoord ).rgb, 1.0 );
```

The uniform uBlocks is the number of mosaic cells across the image and is driven by a slider in the HTML page. Multiplying by uBlocks scales the 0 to 1 texture coordinate up to a grid index, floor() drops the fractional part so neighboring fragments share the same index, adding 0.5 shifts the sample to the center of each cell, and dividing by uBlocks brings the value back into texture space for the texture() call.

Connection to class material

This exercise lines up with what we covered about texture coordinates as per-vertex attributes that get interpolated across the quad. Normally the interpolation is the whole point and gives you a smooth image. Here I use the same interpolated coordinate but modify it inside the fragment shader before sampling, which shows how much visual change is possible from a small amount of math on vTexCoord.